

Fabric_SCD_Type1_Type2_Type3_Watermark_Project

New notebook

Fabric Lakehouse SCD (Type 1 / 2 / 3) + Watermark Incremental Loads

This notebook provides **production-style patterns** for Slowly Changing Dimensions (SCD) in **Microsoft Fabric Lakehouse** using **Spark (PySpark/SQL) over Delta**.

It supports:

- **SCD Type 1** (overwrite)
- **SCD Type 2** (full history with effective dating)
- **SCD Type 3** (current + previous value columns for selected attributes)
- **Incremental loading** using a **watermark column**: `updated_at`
- **Auditability** (run id, ingestion timestamp, source updated timestamp)
- **Dynamic documentation** (auto-generates a Markdown doc into your Lakehouse Files)

Tip: Set `scd_mode` to `"type1"`, `"type2"`, or `"type3"`.

By Olubunmi Obideyi

1) Parameters (edit these)

- `business_keys` : columns that uniquely identify a dimension record in the source (natural key)
- `tracked_columns` : attributes to apply SCD logic to (exclude keys and audit columns)
- `type3_columns` : for Type 3 only — attributes where you want to keep `previous_<col>`

```
In [23]: from pyspark.sql import functions as F
from pyspark.sql import types as T
import uuid
import json
from datetime import datetime

# -----
# REQUIRED PARAMETERS
# -----

# SCD mode: "type1" | "type2" | "type3"
scd_mode = "type2"

# Source / target (Delta) locations (Lakehouse)
source_path = "Files/staging/customers"      # e.g., Files/staging/<entity>
target_path = "Tables/dim_customer"          # Fabric: prefer Tables/<name> for Lakehouse tables
target_table = "dim_customer"                 # Spark table name (optional if you rely purely on path)

# Natural/business key(s)
business_keys = ["customer_id"]

# Watermark column in the SOURCE (created if missing)
source_watermark_col = "updated_at"

# Attributes to track for changes (exclude keys + audit cols)
# If empty, we'll infer as: all cols except business_keys and watermark
tracked_columns = ["customer_name", "customer_status", "address"] # [] # e.g., ["first_name", "last_name"]

# Type3 only: keep previous value for these columns
type3_columns = ["status"] # e.g., ["status", "tier"]
```

```

# Target table SCD/audit columns - columns names
scd_start_col = "effective_start_date"
scd_end_col   = "effective_end_date"      #"effective_end_ts"
is_current_col = "is_current"
row_hash_col  = "row_hash"

# Audit columns
run_id_col = "run_id"
ingestion_ts_col = "ingestion_ts"
source_updated_ts_col = "source_updated_ts"

# Watermark storage
watermark_table = "etl_watermarks"      # a small Delta table in the Lakehouse
pipeline_name = "dim_customer_scd"      # used as key in watermark table + docs
docs_output_path = f"Files/docs/{pipeline_name}.md"

# Incremental loading toggle
incremental = True # True uses watermark; False processes full source

# Safety: if True, will display key counts and samples
debug = True

run_id = str(uuid.uuid4())
ingestion_ts = datetime.utcnow().isoformat(timespec="seconds") + "Z"

```

StatementMeta(, da49657c-ecb3-4056-8df2-72e46790b7fb, 25, Finished, Available, Finished, False)

Create the table that holds the process run log where tracking and scheduling of job runs - if does not exist

```

In [24]: %%sql
CREATE TABLE IF NOT EXISTS etl_run_log (
    batch_id          STRING,
    notebook_name     STRING,
    target_table      STRING,
    start_time        TIMESTAMP,
    end_time          TIMESTAMP,
    status            STRING,
    rows_source       BIGINT,
    rows_inserted     BIGINT,
    rows_updated      BIGINT,
    error_message     STRING
)
USING DELTA;

```

StatementMeta(, da49657c-ecb3-4056-8df2-72e46790b7fb, 26, Finished, Available, Finished, False)

Out[24]: <Spark SQL result set with 0 rows and 0 fields>

We want to retrieve the module name dynamically to track in our audit/log of the SCD run process

- Captured the executing notebook identifier dynamically at runtime using the Spark **environment context**.
- Recorded this metadata within the audit log to ensure execution traceability and governance compliance

```

In [59]: from notebookutils import mssparkutils

ctx = mssparkutils.runtime.context

# print(ctx) - we can get some attributes of the fabric environment
print(ctx.get("currentNotebookName"))

```

StatementMeta(, da49657c-ecb3-4056-8df2-72e46790b7fb, 61, Finished, Available, Finished, False)
Fabric_SCD_Type1_Type2_Type3_Watermark_Docs

audit attributes to log - batch id, module name and target table name

```

In [60]: batch_id = str(uuid.uuid4())
notebook_name = ctx.get("currentNotebookName")      #"e.g scd_type2_lite"
target_table = "dim_customer"

```

Prepare the audit tracking stage for default entries for the process start

```
In [63]: spark.sql(f"""
INSERT INTO etl_run_log
SELECT
    '{batch_id}',
    '{notebook_name}',
    '{target_table}',
    current_timestamp(),
    NULL,
    'RUNNING',
    NULL, NULL, NULL
""")
```

StatementMeta(, da49657c-ecb3-4056-8df2-72e46790b7fb, 65, Finished, Available, Finished, False)

Out[63]: DataFrame[]

2) Helper functions (watermark, hashing, docs)

```
In [30]: def ensure_watermark_table():
    spark.sql(f"""
    CREATE TABLE IF NOT EXISTS {watermark_table} (
        pipeline_name STRING,
        last_watermark TIMESTAMP,
        updated_utc TIMESTAMP
    )
    USING delta
    """)

    ''' Use this when: You need the exact 4th row by some rule
        from pyspark.sql.window import Window from pyspark.sql.functions import
        window = Window.orderBy("updated_utc")
        df_with_rn = df.withColumn("rn", row_number().over(window))

        fourth_row = (
            df_with_rn
            .filter("rn = 4")
            .select("last_watermark")
            .first()
        )

    ...

def get_last_watermark(pipeline_name: str):
    ensure_watermark_table()
    df = spark.sql(f"SELECT last_watermark FROM {watermark_table} WHERE pipeline_name = '{pipeline_name}'")
    rows = df.collect()
    #preference row = df.first() last_watermark = row.last_watermark if row

    return rows[0]["last_watermark"] if rows else None

def set_last_watermark(pipeline_name: str, watermark_ts):
    ensure_watermark_table()
    if watermark_ts is None:
        return
    spark.sql(f"""
    INSERT INTO {watermark_table}
    SELECT '{pipeline_name}' AS pipeline_name,
        TIMESTAMP('{watermark_ts}') AS last_watermark,
        current_timestamp() AS updated_utc
    """)

def ensure_updated_at(df, col_name: str):
    # If source doesn't have updated_at, create it (so incremental mode still works)
    if col_name not in df.columns:
        df = df.withColumn(col_name, F.current_timestamp())
    else:
        # Ensure it's timestamp type
        df = df.withColumn(col_name, F.col(col_name).cast("timestamp"))
```

```

    return df

def infer_tracked_columns(df, business_keys, watermark_col):
    exclude = set(business_keys + [watermark_col, run_id_col, ingestion_ts_col, source_updated_ts_col,
                                     scd_start_col, scd_end_col, is_current_col, row_hash_col])

    return [c for c in df.columns if c not in exclude]

''' --- with_row_hash ----
Typical use cases:
Detect if a row has changed without comparing many columns
SCD Type 2 implementations
Incremental loads / CDC
Deduplication
'''

def with_row_hash(df, cols):
    # Null-safe deterministic hash across tracked columns
    exprs = [F.coalesce(F.col(c).cast("string"), F.lit("0")) for c in cols] # at this point. it's
    return df.withColumn(row_hash_col, F.sha2(F.concat_ws("$", *exprs), 256)) # Now the expressions

```

StatementMeta(, da49657c-ecb3-4056-8df2-72e46790b7fb, 32, Finished, Available, Finished, False)

3) Load source and apply watermark (incremental)

```

In [ ]: df_source = spark.read.format("delta").load(source_path)
df_source = ensure_updated_at(df_source, source_watermark_col)

if not tracked_columns:
    tracked_columns = infer_tracked_columns(df_source, business_keys, source_watermark_col)

# Apply incremental watermark filter
last_wm = get_last_watermark(pipeline_name) if incremental else None # Ternary operator version of
if incremental and last_wm is not None:
    df_source_filtered = df_source.filter(F.col(source_watermark_col) > F.lit(last_wm)) # last_wm
else:
    df_source_filtered = df_source

# Add audit columns to staged source
df_stage = (
    df_source_filtered
    .withColumn(ingestion_ts_col, F.current_timestamp())
    .withColumn(run_id_col, F.lit(run_id))
    .withColumn(source_updated_ts_col, F.col(source_watermark_col)) # Lasted date
)

df_stage = with_row_hash(df_stage, tracked_columns)

if debug:
    print("SCD mode:", scd_mode)
    print("Business keys:", business_keys)
    print("Tracked columns:", tracked_columns)
    print("Last watermark:", last_wm)
    print("Stage count:", df_stage.count())
    display(df_stage.limit(25))

```

4) Ensure target table exists (schema depends on SCD mode)

```

In [33]: def ensure_target(df_stage):
# Build target schema: business keys + tracked cols + scd/audit cols
base_cols = business_keys + tracked_columns

df0 = df_stage.select(*base_cols) # all the columns tracked for changes

if scd_mode == "type3":
    # Add previous_col columns
    for c in type3_columns: # columns that should be flagged with
        if c in df0.columns:
            df0 = df0.withColumn(f"previous_{c}", F.lit(None).cast(df0.schema[c].dataType))
        else:

```

```

        raise ValueError(f"Type3 column '{c}' not found in source/tracked columns.")

# Add SCD & audit cols
if scd_mode in ("type2",):
    df0 = (df0
            .withColumn(scd_start_col, F.lit(None).cast("timestamp"))
            .withColumn(scd_end_col, F.lit(None).cast("timestamp"))
            .withColumn(is_current_col, F.lit(True).cast("boolean"))
            .withColumn(row_hash_col, F.lit(None).cast("string"))
            )

# Always keep audit cols (useful for type1/type3 too)
df0 = (df0
        .withColumn(run_id_col, F.lit(None).cast("string"))
        .withColumn(ingestion_ts_col, F.lit(None).cast("timestamp"))
        .withColumn(source_updated_ts_col, F.lit(None).cast("timestamp"))
        )

# Create if missing
if not spark._jsparkSession.catalog().tableExists(target_table):
    (df0.limit(0)                                     # Replicate a table schema from an empty dataframe (zeor value
     .write.format("delta")
     .mode("overwrite")
     .option("overwriteSchema", "true")
     .saveAsTable(target_table))
    print(f"Created target table: {target_table}")
else:
    print(f"Target table exists: {target_table}")

ensure_target(df_stage)

```

StatementMeta(, da49657c-ecb3-4056-8df2-72e46790b7fb, 35, Finished, Available, Finished, False)
Target table exists: dim_customer

5) Run SCD merge (Type 1 / 2 / 3)

```

In [ ]: from delta.tables import DeltaTable

target = DeltaTable.forName(spark, target_table)

# Common sets
key_join = " AND ".join([f"t.{k} = s.{k}" for k in business_keys])

if debug:
    print(key_join)

def scd_type1_merge():
    # Overwrite attributes in place
    update_set = {c: f"s.{c}" for c in tracked_columns}
    # audit columns
    update_set.update({
        run_id_col: f"s.{run_id_col}",
        ingestion_ts_col: f"s.{ingestion_ts_col}",
        source_updated_ts_col: f"s.{source_updated_ts_col}",
        row_hash_col: f"s.{row_hash_col}",
    })

    insert_cols = business_keys + tracked_columns + [row_hash_col, run_id_col, ingestion_ts_col, source_updated_ts_col]
    insert_set = {c: f"s.{c}" for c in insert_cols} # we now create the dictionary that is

    if debug:
        print(insert_cols)

    (target.alias("t")
     .merge(df_stage.alias("s"), key_join)
     .whenMatchedUpdate(set=update_set)
     .whenNotMatchedInsert(values=insert_set)
     .execute())

def scd_type2_merge():

```

```

if debug:
    df_test = target.toDF()
    print("df_target columns : ")
    #print(df_test)
    display(df_test)

print("Call before filtering target table 0")
print(f"Row has is : {row_hash_col}")
# Type 2: expire current row + insert new version on change
df_t = target.toDF().filter(F.col(is_current_col) == True).select(*business_keys, row_hash_col) #
#

print("1")
df_s = df_stage.select(*business_keys, *tracked_columns, row_hash_col, run_id_col, ingestion_ts_col)
print("2")
# Identify changed keys (hash differs) and new keys
df_join = (df_s.alias("s")
            .join(df_t.alias("t"), on=business_keys, how="left"))
print("3")
if debug:
    print("Start")
    print(df_join)

print("4")
df_changes = df_join.filter((F.col("t."+row_hash_col).isNotNull()) & (F.col("s."+row_hash_col) !=
df_new = df_join.filter(F.col("t."+row_hash_col).isNull()))

# Staged merge dataset technique:
# - rows to EXPIRE: merge_key = business_key (match existing current row)
# - rows to INSERT: merge_key = NULL (forces NOT MATCHED, inserts new version)
# We'll use an additional column 'merge_key' to control match behavior.
print("5")
expire_rows = (df_changes
                .select("s.*")
                .withColumn("merge_key", F.col(business_keys[0]).cast("string")))

print("6")
insert_changed_rows = (df_changes
                       .select("s.*")
                       .withColumn("merge_key", F.lit(None).cast("string")))

print("7")
insert_new_rows = (df_new
                   .select("s.*")
                   .withColumn("merge_key", F.lit(None).cast("string")))

print("8")
staged = expire_rows.unionByName(insert_changed_rows).unionByName(insert_new_rows)

print("9")
# Add SCD columns for inserts (start/end/current)
now_ts = F.current_timestamp()
staged = (staged
          .withColumn(scd_start_col, now_ts)
          .withColumn(scd_end_col, F.lit(None).cast("timestamp"))
          .withColumn(is_current_col, F.lit(True)))

print("10")
# Build merge condition: match only when merge_key present and keys match and target current
merge_cond = " AND ".join([f"t.{k} = s.{k}" for k in business_keys]) + f" AND t.{is_current_col} ="

print("Merge_cond sql parameter")
if debug:
    print(merge_cond)

print("updateset")
# Update when matched: expire current
update_set = {
    is_current_col: "false",
    scd_end_col: "current_timestamp()",
    run_id_col: f"s.{run_id_col}",
    ingestion_ts_col: f"s.{ingestion_ts_col}",
    source_updated_ts_col: f"s.{source_updated_ts_col}",
}

```

```

print(update_set)

insert_cols = business_keys + tracked_columns + [row_hash_col, scd_start_col, scd_end_col, is_curr
run_id_col, ingestion_ts_col, source_updated_ts_c

insert_set = {c: f"s.{c}" for c in insert_cols}

if debug:
    print("insert_set")
    print(insert_set)
    print("staged")
    display(staged)
    print("target")
    display(target.toDF())

(target.alias("t")
 .merge(staged.alias("s"), merge_cond)
 .whenMatchedUpdate(set=update_set)
 .whenNotMatchedInsert(values=insert_set)
 .execute())

def scd_type3_merge():
    # Type 3: for selected columns, shift current -> previous on change
    # We'll merge on keys and update previous_col and col when changed
    # Change detection uses hash across tracked_columns (or you can limit to type3_columns)
    update_set = {c: f"s.{c}" for c in tracked_columns}

    # For each type3 column: previous_col = t.col, and t.col = s.col
    for c in type3_columns:
        update_set[f"previous_{c}"] = f"t.{c}"
        update_set[c] = f"s.{c}"

    update_set.update({
        run_id_col: f"s.{run_id_col}",
        ingestion_ts_col: f"s.{ingestion_ts_col}",
        source_updated_ts_col: f"s.{source_updated_ts_col}",
        row_hash_col: f"s.{row_hash_col}",
    })

    insert_cols = business_keys + tracked_columns + [f"previous_{c}" for c in type3_columns] + [row_ha
    # For inserts, previous_* are null
    insert_set = {c: f"s.{c}" for c in business_keys + tracked_columns + [row_hash_col, run_id_col, ir
    for c in type3_columns:
        insert_set[f"previous_{c}"] = "NULL"

    # Only update when hash differs (otherwise no-op)
    cond_change = f"t.{row_hash_col} IS NULL OR t.{row_hash_col} <> s.{row_hash_col}"

    (target.alias("t")
     .merge(df_stage.alias("s"), key_join)
     .whenMatchedUpdate(condition=cond_change, set=update_set)
     .whenNotMatchedInsert(values=insert_set)
     .execute())

# Execute
if df_stage.count() == 0:
    print("No new rows to process. Exiting.")
else:
    if scd_mode == "type1":
        scd_type1_merge()
    elif scd_mode == "type2":
        scd_type2_merge()
    elif scd_mode == "type3":
        scd_type3_merge()
    else:
        raise ValueError("scd_mode must be one of: type1, type2, type3")

```

```
# Update watermark to max(updated_at) from processed stage
max_wm = df_stage.select(F.max(F.col(source_watermark_col)).alias("m")).collect()[0]["m"]
if incremental:
    set_last_watermark(pipeline_name, str(max_wm))
    print("Updated watermark to:", max_wm)

if debug:
    print("Target sample:")
    display(spark.table(target_table).orderBy(F.col(ingestion_ts_col).desc()).limit(50))

# Olubunmi Obideyi
```